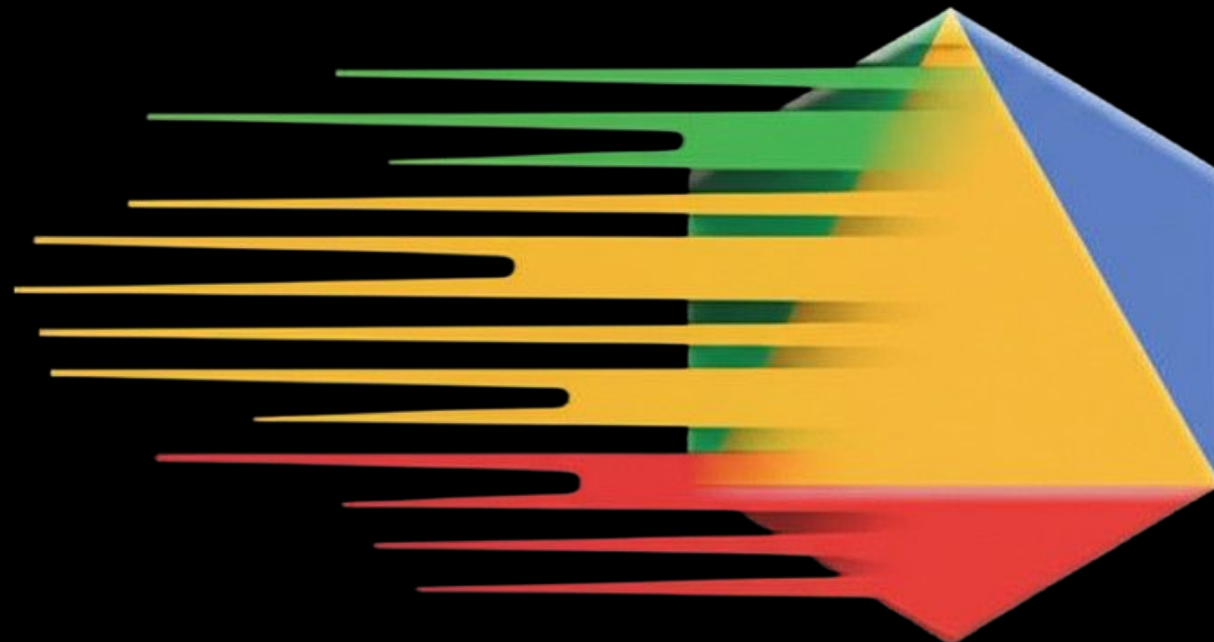


Tech @ Google

Optimiser un algo pour la rapidité

Code : **Bruno De Backer**
Présentation : **Thibaut Cuvelier**
Google France (Operations Research)



Recherche opérationnelle, késako ?



- 01 **Ordonnancement** (scheduling)
- 02 **Planification** (planning)
- 03 **Routage** (routing)
- 04 **Assignation** (assignment)
- 05 **Conditionnement** (packing)

L'équipe recherche opérationnelle



Routage : logistique, exploration avec Google Street View

- Solveur libre
- Produit B2B : GMPRO (API)

Problèmes pratiques et concrets :

- Planification du personnel (API)
- Conception de réseaux de distribution (API)

Solveurs de bas niveau :

- Glop : solveur LP à base de simplexe
- CP-SAT : solveur CP à base de SAT, qui a gagné 10+ médailles d'or à la compétition MiniZinc
- PDLP : solveur LP de premier ordre
- MathOpt : couche de modélisation mathématique

Un produit libre : OR-Tools

Optimiser pour la rapidité

01 Problème de couverture

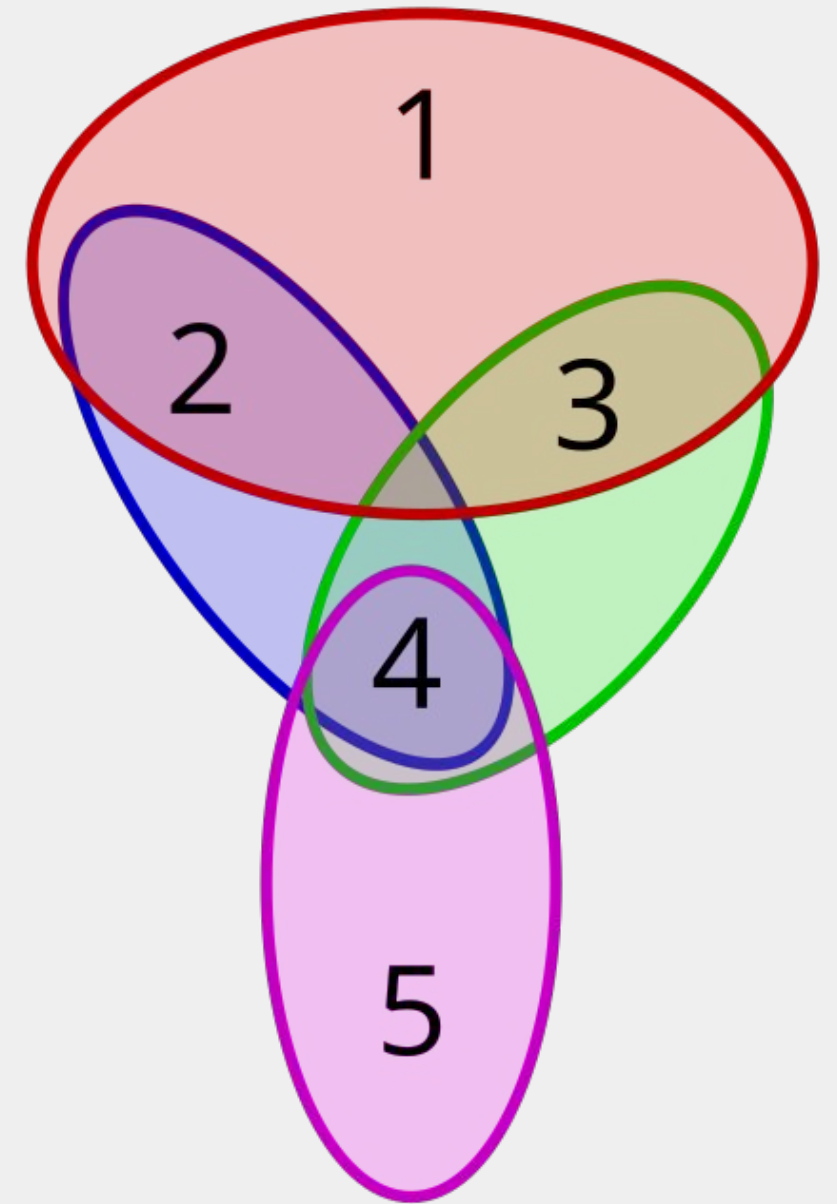
02 Algorithme de Chvátal

03 Implémentation

04 Amélioration de la rapidité

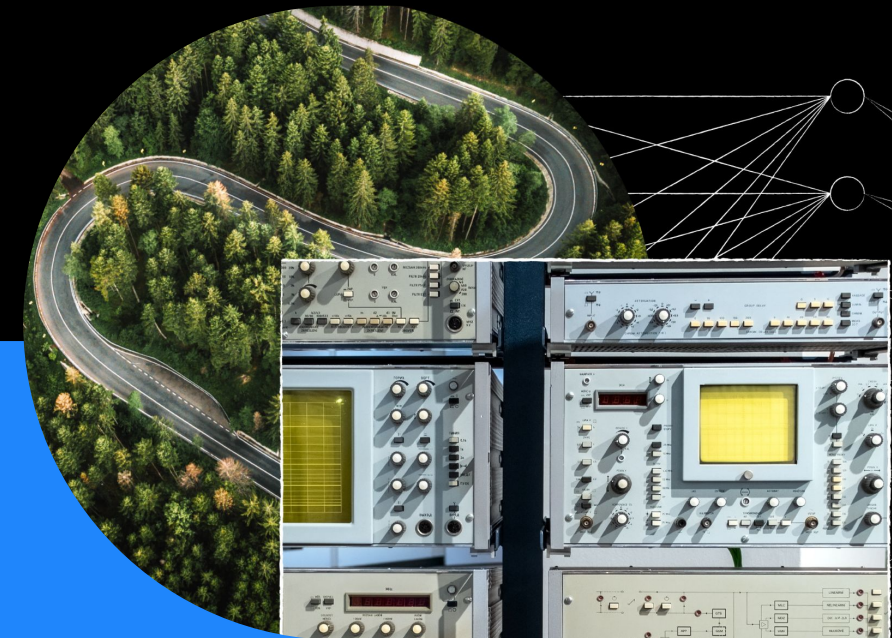
05 Outils

Couverture



À quoi ça sert ?

- **Fuzzing** : minimiser le nombre de tests en couvrant tout le code
- **GenAI** : choisir aussi peu de données que possible pour couvrir tous les domaines d'intérêt pour entraîner un modèle
- **Sécurité physique** : installer le minimum de caméras pour couvrir tous les couloirs d'un bâtiment
- **Logistique** : couvrir les livraisons à effectuer par les routes des camions

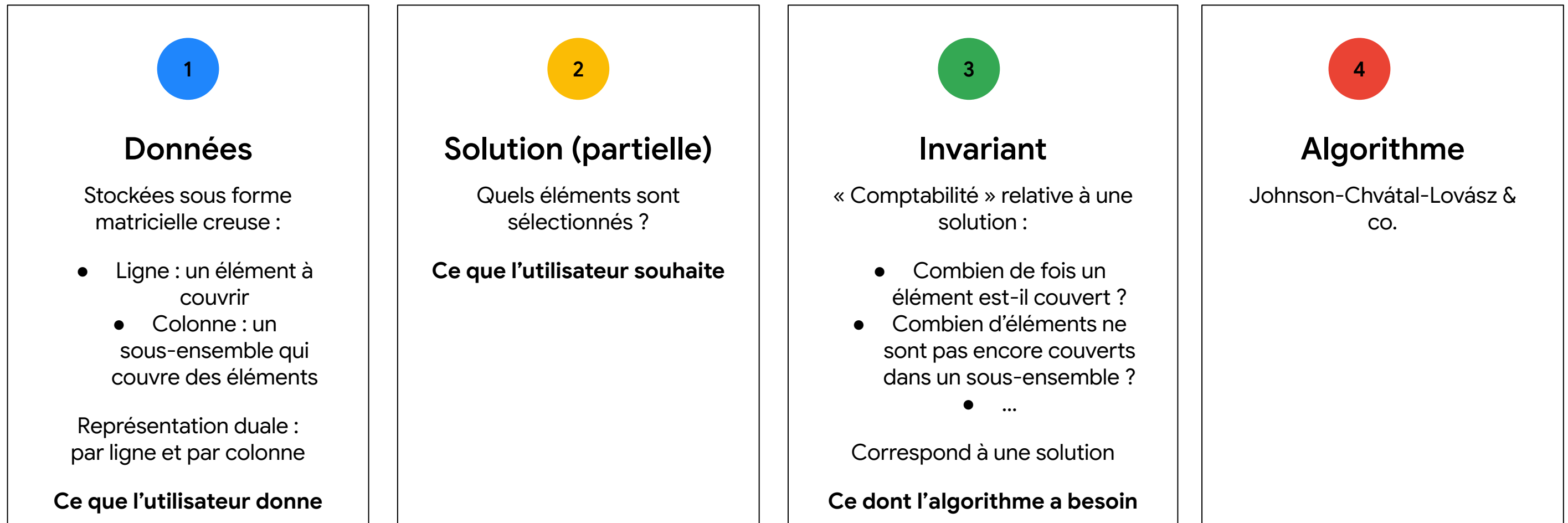


Algorithme de Johnson-Chvátal-Lovász

- **Tant que tous les éléments ne sont pas couverts :**
 - Prendre le sous-ensemble avec le meilleur ratio coût par nombre d'éléments nouvellement couverts
- Algorithme glouton : idéal pour la rapidité
- Meilleur algorithme rapide dans la littérature depuis les années 1970
- Notre cas d'utilisation : logistique
 - X millions de colis (éléments)
 - X milliards de chemins possibles (sous-ensembles)
 - Réponse en quelques secondes



Première implémentation : séparation des responsabilités



Structures de données réutilisables

Et la performance ?

60 Mo

Taille de l'instance de test
(rail4284, Mittelman) :

- 4284 éléments
- 1 096 894 sous-ensembles
- 11 284 032 valeurs dans la matrice

5^s

Temps d'exécution de la première
implémentation

File à priorité adaptable pour sélectionner le
prochain sous-ensemble

https://github.com/google/or-tools/blob/stable/ortools/base/adjustable_priority_queue.h

Performance des implémentations
académiques

200 ms

Temps d'exécution avec une autre file à
priorité (arité k)

Optimisée pour changer la priorité souvent

https://github.com/google/or-tools/blob/main/ortools/algorithms/adjustable_k_ary_heap.h

https://github.com/google/or-tools/blob/main/ortools/set_cover/set_cover_heuristics.h :

GreedySolutionGenerator

10 fois plus rapide

Et après ?

1

Est-ce utile ?

Faut-il vraiment mettre à jour la priorité ?

Sélectionner un sous-ensemble : mettre à jour la valeur pour tout sous-ensemble qui a au moins un élément en commun

Ne peut-on pas juste la calculer au vol ?

2

Est-ce justifié ?

Sélectionner un sous-ensemble selon son coût : est-ce une bonne décision ?

Et si on sélectionnait plutôt des éléments ?

D'abord le plus contraint : s'il n'y a qu'une manière de le couvrir, on ne peut pas prendre une mauvaise décision au début...



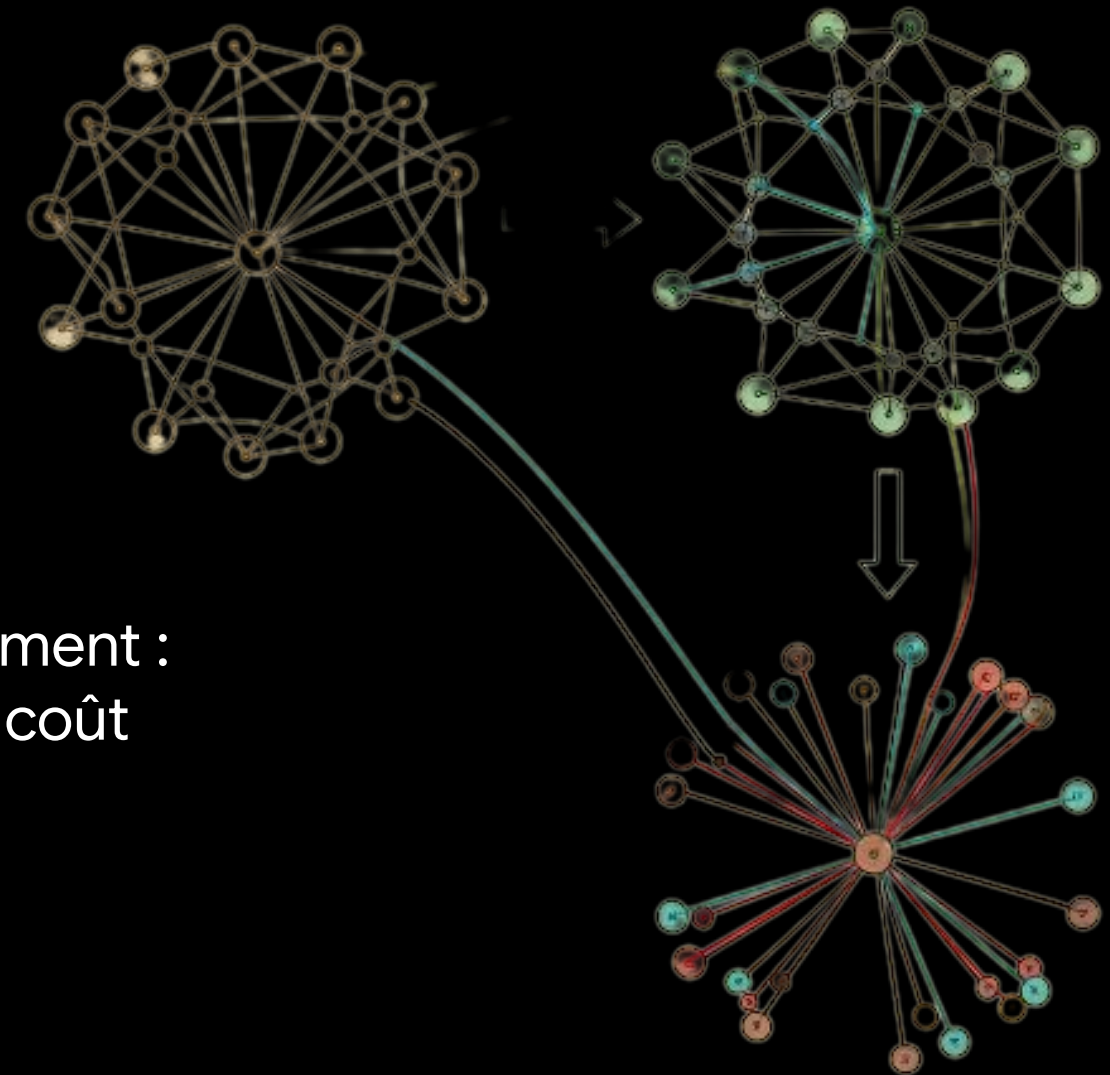
Tri selon les degrés

Nouvelle heuristique !

L'algorithme :

1. Trier les éléments selon leur degré (nombre de sous-ensembles pouvant les couvrir)
2. Itérer sur les éléments par degré croissant :
 - a. Itérer sur tous les sous-ensembles couvrant cet élément :
 - i. Prendre le sous-ensemble avec le meilleur ratio coût par nombre d'éléments nouvellement couverts

L'opération la plus chère ? Transposer la matrice !



Et la performance ?

1504 Mo

Taille de l'instance de test
(FIMI webdocs) :

- 5 267 656 éléments
- 1 692 082 sous-ensembles

<http://fimi.uantwerpen.be/data/>

23 000 s

Temps d'exécution de la première
implémentation de Chvátal

https://github.com/google/or-tools/blob/main/ortools/set_cover/set_cover_heuristics.h :
GreedySolutionGenerator

Performance des implémentations
académiques

500 ms

Temps d'exécution avec la nouvelle
heuristique

https://github.com/google/or-tools/blob/main/ortools/set_cover/set_cover_heuristics.h :
ElementDegreeSolutionGenerator

46 000 fois plus rapide

Comment y arriver ?

Un principe de base : peu d'indirection, de pointeurs

Des optimisations majeures :

- **Pas de file à priorité**, surtout avec des modifications de priorité à chaque itération — la différence majeure entre les deux heuristiques
- **Structures de données creuses... et denses**
- **Tri par base** plutôt que `std::sort`

Des microoptimisations :

- **Comparer deux sous-ensembles** : ratios coût sur élément nouvellement couverts. Pas besoin de diviser !
- **Vérifier comment le code est compilé** : utiliser les dernières instructions (par exemple, ABM sur x86)

Complexité finale : $O(\text{nnz})$, avec nnz nombre d'éléments dans la matrice (expérimentale)

Comment y arriver ?



Outils :

- **Données de test, benchmark sérieux**, puis génération d'instances plus grandes (c'est très lent de générer un nombre aléatoire !)
- **Profilage de code** : par exemple, on a pu déterminer que la file à priorité dominait dans la première implémentation
- **Instrumentation manuelle du code** : déterminer si on utilise une valeur calculée (ComputationUsefulnessStats)
- **Analyse de l'assembleur généré** : Compiler Explorer (Godbolt), comparer GCC et Clang — instructions récentes, vectorisation

Et après ?

Comment faire mieux en performance ?

- **La dernière limite : la mémoire**
- Pas assez de bande passante entre RAM et CPU !
- Compresser les données : perte d'un facteur 4 en performance, gain d'un facteur 2 en mémoire

Quelles fonctionnalités manquent ?

- Contraintes supplémentaires, comme la capacité
- Solutions de meilleure qualité

Et l'impact ?

- Sans ce travail, tout un projet de logistique tomberait à l'eau
 - On ne pouvait pas se permettre d'attendre 6 h pour un résultat !
- Brique de base pour de nouveaux algorithmes (par exemple, dans CP-SAT)

Et les limitations ?

- Grâce à ce travail, très peu de travail à effectuer par élément en mémoire
- Inutile de mettre cet algorithme derrière une API : le réseau serait une limite

Merci !